COMPANION TO THE RAC ON KUBEADM LAB GUIDE

The StatefulSet, line by line

Guide step 10.4 turns two RAC servers into two StatefulSets. Every field of that manifest, why it is there, where Grid uses it, what happens when you apply it, what survives a restart, and how to see each one on the running cluster.

`StatefulSet` `nodeAffinity` `sysctls · capabilities` `hugepages · QoS` `volumeDevices · hostPath` `systemd · preStop`

25 Oct 2026 · The lab follows Oracle's How to Install and Configure Oracle RAC on Oracle Cloud Native Environment. This page explains guide step 10.4 in depth.

`PowerShell`

on your PC

`worker · controlplane`

root shell on a VM

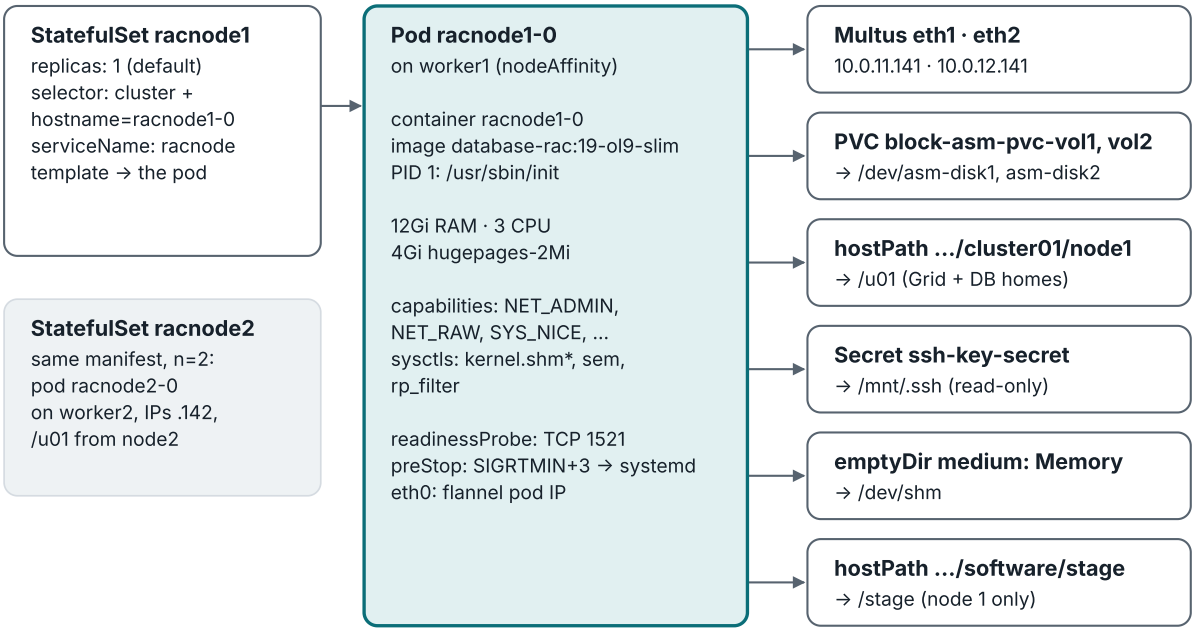
`racnode1-0`

root shell in a RAC pod

At a glance

Guide step 10.4 runs one loop that creates **two StatefulSets**, `racnode1` and `racnode2`. Each StatefulSet creates and looks after exactly **one pod**: `racnode1-0` on worker1 and `racnode2-0` on worker2. Those two pods are the two RAC "servers". Everything Grid Infrastructure would normally get from a physical server (a fixed host name, its own disks, the shared ASM disks, two private network cards, kernel settings, a full init system) is written into the StatefulSet's **pod template**.

One StatefulSet, one pod, and everything the pod is wired to



The ASM PVCs are the same two claims in both StatefulSets: that is how both pods see the same disks.
The hostPath volumes are per worker: /u01 on worker1 is node1, on worker2 node2.

racnode1 shown; racnode2 is the same with worker2, .142 and node2. Each box on the right is one entry in volumes or the Multus annotation.

The manifest has four layers. This guide goes through them in the same order:

LAYER	FIELDS	WHAT IT DECIDES
The StatefulSet itself	`metadata`, `selector`, `serviceName`, `replicas` (left at 1)	the object's name, which pods it owns, the pod's name and host name
The pod	`labels`, the Multus annotation, `securityContext.sysctls`, `affinity`, `volumes`	where the pod runs, its networks, kernel settings, and what storage it can use
The container	`image`, `command`, `resources`, `securityContext.capabilities`, `lifecycle`	what runs, how much CPU, memory and hugepages it gets, which privileges, how it shuts down
The container's use of the volumes	`volumeDevices`, `volumeMounts`, `readinessProbe`	where each disk and directory appears inside the pod, and when the pod counts as Ready

Two StatefulSets, not one with two replicas. A single StatefulSet with `replicas: 2` would also give the names `racnode-0` and `racnode-1`. But every pod of one StatefulSet gets the same template, and the RAC pods must differ: each needs its own worker, its own `/u01` directory, its own private IPs (.141 and .142), and only node 1 mounts the installation zips. One StatefulSet per node keeps each pod's settings explicit. The Oracle doc does the same.

Where you meet these objects later:

- Phase 11 installs Grid **inside** these pods (`kubectl -n rac exec -it racnode1-0 -- bash`).
- Phase 13's NodePort Services and Phase 10.3's headless Services find the pods through the labels set here.
- Phase 14 deletes a pod on purpose; the StatefulSet recreates it with the same name, on the same worker, with the same disks.

Kubernetes background in five points

1. A pod is the unit Kubernetes runs. A pod is one or more containers that share a network address, a host name and a set of volumes. Our RAC pods have one container each. Kubernetes never "repairs" a pod: if it dies, a **new** pod is made from a template. Anything written only inside the container's own filesystem is lost then; only volumes survive (guide Phase 14 deals with that).

2. You rarely create pods directly; a controller does. A controller is a loop in the control plane that compares what you asked for (the spec) with what exists, and fixes the difference. You write "I want this pod to exist"; the controller creates it, and recreates it whenever it disappears.

3. Deployment versus StatefulSet. Both are controllers that make pods from a template. They differ in what they promise about each pod:

	DEPLOYMENT	STATEFULSET
Pod name	random suffix: `web-7d9f8c6b5-x2kqp`	fixed ordinal: `racnode1-0`, every time
Host name inside the pod	the random name	the fixed name
After a pod is deleted	a new pod with a **new** name	a new pod with the **same** name
Two copies at once?	allowed during updates	never: the old pod must be gone before the new one starts
Per-pod storage	all replicas share one template	can give each ordinal its own claim (`volumeClaimTemplates`)
Typical use	stateless web servers	databases, queues, anything with an identity

4. A PersistentVolumeClaim (PVC) is a pod's request for storage. A PersistentVolume (PV) is the actual storage, here one shared VirtualBox disk on both workers (guide Phase 7). A PVC is a claim on a PV. The pod only names the PVC; Kubernetes finds the PV behind it. That indirection is why the StatefulSet doesn't contain any disk ID.

5. Labels connect everything. A StatefulSet finds "its" pods by a label `selector` . Services find pods the same way. Labels are free-form `key: value` pairs; the RAC pods carry `cluster: racnode` and `hostname: racnode1-0` (or `racnode2-0`), and Kubernetes adds `statefulset.kubernetes.io/pod-name: racnode1-0` by itself.

Why RAC needs a StatefulSet

Grid Infrastructure was written for physical servers. It assumes four things about each node, and a StatefulSet is the Kubernetes object that provides all four.

GRID ASSUMES	WHY IT MATTERS TO GRID	WHAT PROVIDES IT
The host name never changes	The installer records `racnode1-0` and `racnode2-0` as the cluster's node names in the OCR (the cluster registry on ASM) and the OLR (each node's local registry under `/u01`). A node that boots with another name is a stranger to the cluster.	StatefulSet pod name = host name, always `<statefulset name>-0`
The node's own disk comes back	`/u01` holds the Grid home, the database home, the inventory and the OLR.	`hostPath` volume plus `nodeAffinity`: the pod always lands on the worker that has its directory
The shared disks are the same on both nodes	ASM's disk group `DATA` holds the OCR, the voting files and the database.	the same two PVCs named in both StatefulSets
There is never a second copy of a node	Two processes both acting as racnode1-0 against the same voting files and data files would corrupt the cluster.	the StatefulSet guarantee: at most one pod per name at any time

Why not just a bare Pod? A pod created on its own (`kind: Pod`) is not recreated when it is deleted or its worker reboots. The StatefulSet controller does that, and Phase 14 relies on it.

Why not a Deployment? Each restart would give a new random name such as `racnode-5c8f7d9b4-q7tzl` ; Grid would see a new, unknown node. A Deployment may also run the old and the new pod side by side for a moment during an update, which RAC must never do.

Why no `volumeClaimTemplates` **?** That's the usual StatefulSet way to give each pod its **own** disk: Kubernetes creates a claim per pod, such as `data-racnode1-0` . ASM disks are the opposite: **one** disk used by **both** pods. So the claims are created once by hand in guide Phase 7 (`block-asm-pvc-vol1` , `block-asm-pvc-vol2`) and both StatefulSets name the same two claims. Each node's private disk (`/u01`) is a `hostPath` directory rather than a claim, because it is plain local storage that only one worker ever uses.

Why one StatefulSet per node, with `replicas: 1` **?** Covered in "At a glance": the two pods differ in their worker, `/u01` directory, private IPs and the `/stage` mount, and one template can't express that. A side benefit: you can stop, restart or change one RAC node without touching the other's object.

The manifest, field by field

Each part below quotes a piece of the 10.4 manifest for `n=1` , then says why it is there, where it is used, and what happens without it.

The shell loop around the YAML

```shell
for n in 1 2; do
  stagevol=""; stagemnt=""
  if [ $n = 1 ]; then
    stagevol=$'       - name: stagevol\n         hostPath: { path: /scratch/software/stage }'
    stagemnt=$'         - { mountPath: /stage, name: stagevol }'
  fi
  cat <<EOF | kubectl apply -f -
  ...
EOF
done
```

- `for n in 1 2` : the same text is applied twice. Every `$n` inside becomes `1` , then `2` : `racnode$n` → `racnode1` , `worker$n` → `worker1` , `10.0.11.14$n` → `10.0.11.141` .
- `<<EOF` **without quotes:** the shell expands `$n` , `$stagevol` and `$stagemnt` before `kubectl` sees the text. (Phase 10.3's SCAN block uses `<<'EOF'` with quotes because it has nothing to fill in.)
- `$'...\n...'` : a bash string in which `\n` is a real line break. `$stagevol` is two YAML lines with the exact indentation of the `volumes:` list, so on node 1 it becomes one more list entry; on node 2 it is empty and leaves a blank line, which YAML ignores.
- `kubectl apply -f -` : read the manifest from standard input and create the object, or update it if it already exists. Running the unchanged loop again is harmless. Running it **with a change** restarts the pods; see "updateStrategy" below.
- **Indentation:** under `template.spec` the manifest indents by 5 spaces, and some lists by 1 or 2. YAML only needs each level to be consistent, so this is valid, just unusual. Copy it exactly.

apiVersion, kind and metadata

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: racnode1
  namespace: rac
  labels: { cluster: racnode, hostname: racnode1-0 }
```

- `apps/v1` / `StatefulSet` : which API and object type. StatefulSets live in the `apps` group, like Deployments.
- `name: racnode1` : the StatefulSet's name. Its pods are named `<name>-<ordinal>` , so with one replica the only pod is `racnode1-0` . That is where the RAC host name comes from; the whole cluster depends on this one field.
- `namespace: rac` : the same namespace as the PVCs (Phase 7), the Secret (10.2) and the Services (10.3). A pod can only use claims and secrets in its own namespace.
- `labels` on the StatefulSet object itself: only for finding it (`kubectl -n rac get sts -l cluster=racnode`). They don't select anything.

spec: selector, serviceName and the defaults you don't see

```yaml
spec:
  selector:
    matchLabels: { cluster: racnode, hostname: racnode1-0 }
  serviceName: racnode
  template:
    ...
```

- `selector.matchLabels`: tells the controller which pods belong to this StatefulSet. It must match the template's labels exactly or the API server rejects the manifest. It can't be changed after creation. The doc uses only `cluster: racnode` for **both** StatefulSets, so each would also "match" the other's pod; Kubernetes warns against overlapping selectors. Adding `hostname: racnode1-0` makes each selector match exactly one pod.
- `serviceName: racnode`: names the "governing" headless Service. The controller copies it into the pod as `subdomain`, and the kubelet then writes the full name `racnode1-0.racnode.rac.svc.cluster.local` into the pod's `/etc/hosts`. No Service called `racnode` exists in this lab, so that full name isn't in DNS; the names Grid uses come from the five Services of 10.3 instead (see the Services guide). The field is kept because the doc has it.
- `replicas` is not set, so it is **1**: one pod per StatefulSet.
- `podManagementPolicy` is not set: `OrderedReady`. It only matters with several replicas, so it has no effect here.
- `updateStrategy` is not set: `RollingUpdate`. **Any change to the template** (an image tag, a memory size, a new mount) makes the controller **delete** `racnode1-0` **and create it again**. That's a RAC node restart. Re-running 10.4 with a change restarts both StatefulSets' pods almost at the same time: a full cluster outage. Change one StatefulSet at a time, and only when you mean to restart that node.

The pod's labels and the Multus annotation

```yaml
  template:
    metadata:
      labels: { cluster: racnode, hostname: racnode1-0 }
      annotations:
        k8s.v1.cni.cncf.io/networks: '[
          {"name":"macvlan-conf1","namespace":"multus-rac","interface":"eth1","ips":["10.0.11.141/24"]},
          {"name":"macvlan-conf2","namespace":"multus-rac","interface":"eth2","ips":["10.0.12.141/24"]}
]'
```

- `labels`: copied onto the pod. They are how everything else finds it:

LABEL	USED BY
`cluster: racnode`	the StatefulSet selector; Service `racnode-scan` (both pods, 10.3); NodePort `racnode-scan-lsnr` (13.3)
`hostname: racnode1-0`	the StatefulSet selector; NodePort `racnode1-np` (13.3)

LABEL	USED BY
`statefulset.kubernetes.io/pod-name: racnode1-0` (added by Kubernetes)	the node and VIP Services `racnode1-0`, `racnode1-0-vip` (10.3)

- **The Multus annotation:** by default a pod gets one network card, `eth0`, from flannel. RAC needs a private interconnect, ideally two. Multus reads this annotation when the pod is created and adds `eth1` and `eth2` using the NetworkAttachmentDefinitions `macvlan-conf1` and `macvlan-conf2` from Phase 6. Each is a macvlan sub-interface of a worker NIC on one private network.
- `"ips"`**:** fixed addresses, `.141` for node 1 and `.142` for node 2. Grid records the interconnect **subnets** at install time and the nodes must reach each other there; fixed IPs keep that predictable across pod restarts. Without the annotation the pod would have no `eth1` / `eth2` and the Grid installer would find no private network.

Pod securityContext: the kernel settings

```yaml
spec:
 securityContext:
   sysctls:
    - { name: kernel.shmall, value: "2097152" }
    - { name: kernel.sem, value: "250 32000 100 128" }
    - { name: kernel.shmmax, value: "8589934592" }
    - { name: kernel.shmmni, value: "4096" }
    - { name: net.ipv4.conf.all.rp_filter, value: "2" }
```

On a normal server these lines would sit in `/etc/sysctl.d/`, put there by `oracle-database-preinstall-19c`. In a container that file doesn't help: `kernel.shm*`, `kernel.sem` and the `net.*` settings belong to the pod's own IPC and network **namespaces**, which Kubernetes creates fresh for every pod. So they are set per pod, here.

SETTING	VALUE	MEANING	USED BY
`kernel.shmmax`	8589934592	largest single shared-memory segment: 8 GiB	the SGA, which is one or more System V segments
`kernel.shmall`	2097152	total shared memory in 4 KiB pages: 8 GiB	all SGAs in the pod together (ASM and the database)
`kernel.shmmni`	4096	maximum number of segments	many instances and processes
`kernel.sem`	250 32000 100 128	per set, total, per call, number of sets	Oracle processes coordinate through semaphores
`net.ipv4.conf.all.rp_filter`	2	"loose" reverse-path filter	Oracle requires 0 or 2 with several private interfaces; with "strict" (1) the kernel may drop interconnect packets that arrive on a different card than the return route

These sysctls are on Kubernetes' "unsafe" list, so the kubelet refuses the pod (`SysctlForbidden`) unless they are allowed in its configuration. That's `allowedUnsafeSysctls` in guide 5.1. They affect only this pod, not the worker.

affinity: always the same worker

```yaml
    affinity:
      nodeAffinity:
        requiredDuringSchedulingIgnoredDuringExecution:
          nodeSelectorTerms:
          - matchExpressions:
            - { key: kubernetes.io/hostname, operator: In, values: [worker1] }
```

- **What:** the scheduler may place this pod **only** on the node whose `kubernetes.io/hostname` label is `worker1` . Every node has that label automatically.
- **Why:** `/u01` is a directory on worker1's disk (next section), and the macvlan interfaces sit on worker1's NICs. On another worker the pod would start with an **empty** `/u01` : no Grid home, no OLR. It also keeps the two RAC nodes on different workers.
- **"required...":** a hard rule. If worker1 is down, the pod stays `Pending` rather than starting elsewhere. That's intended: the surviving node carries the database.
- **"...IgnoredDuringExecution":** if the label changed later, a running pod would not be evicted.

volumes: what storage the pod may use

```yaml
    volumes:
      - name: myshm
        emptyDir: { medium: Memory }
      - name: swvol-racnode1
        hostPath: { path: /scratch/rac/cluster01/node1 }
      - name: asmvol1
        persistentVolumeClaim: { claimName: block-asm-pvc-vol1 }
      - name: asmvol2
        persistentVolumeClaim: { claimName: block-asm-pvc-vol2 }
      - name: sshkeys
        secret: { secretName: ssh-key-secret }
      - name: stagevol                                  # node 1 only
        hostPath: { path: /scratch/software/stage }
```

`volumes` only **declares** storage and gives each a name. Where it appears inside the container is decided later by `volumeMounts` and `volumeDevices` .

VOLUME	TYPE	BECOMES	WHY	SURVIVES A POD RESTART?
`myshm`	`emptyDir`, `medium: Memory` (a RAM disk)	`/dev/shm`	a container's `/dev/shm` is only 64 MB. ASM uses Automatic Memory Management by default, which keeps its memory in `/dev/shm` files, and Grid's checks want it larger. Its contents count against the pod's 12 GiB memory limit.	no, and it doesn't need to
`swvol-racnode1`	`hostPath`	`/u01`	the Grid and database homes, the inventory, the OLR, and the lab's restart state (Phase 14). A plain directory on worker1's `/scratch` disk, created in guide Phase 4.	**yes**: it's on the worker
`asmvol1`, `asmvol2`	PVC (`persistentVolumeClaim`)	`/dev/asm-disk1`, `/dev/asm-disk2`	the shared ASM disks; claims from Phase 7, Block mode. The same two claims are in both StatefulSets.	**yes**
`sshkeys`	`secret`	`/mnt/.ssh`	the key pair from 10.2. Guide 11.1 copies it into the grid and oracle users' `~/.ssh`.	yes (it's in the API server)
`stagevol`	`hostPath`	`/stage`	the Grid and database zip files. Only node 1 runs the installers, so only node 1 mounts it.	yes

The container: name, image, pull policy and command

```yaml
    containers:
      - name: racnode1-0
        image: localhost/oracle/database-rac:19-ol9-slim
        imagePullPolicy: IfNotPresent
        command: [/usr/sbin/init]
```

- `name`: the container's name inside the pod. `kubectl exec` picks it automatically because there's only one.
- `image`: built on each worker in guide Phase 8 and stored in that worker's CRI-O storage. `localhost/` means it came from no registry.
- `imagePullPolicy: IfNotPresent`: use the local image if the worker has it. The doc's `Always` would try to download `localhost/oracle/...` from a registry, fail, and leave the pod in `ErrImagePull`.
- `command: [/usr/sbin/init]`: run **systemd** as process 1, as on a real server. Grid installs a systemd service (`oracle-ohasd`) that starts the cluster stack at boot, and expects `sshd`, `rc.local` and other services. A normal container runs one program; this one runs a whole init system. On each start systemd runs `rc.local`, which runs the image's `setupContainerEnv.sh` (Phase 8): it gives `/dev/asm-disk*` to `grid:asmadmin`, mode 660, and calls the restart hook from Phase 14.

resources: CPU, memory and hugepages

```yaml
        resources:
          requests: { memory: "12Gi", cpu: "3" }
          limits: { hugepages-2Mi: "4Gi", memory: "12Gi", cpu: "3" }
```

- `requests`: what the scheduler reserves. Worker1 must have 3 CPUs and 12 GiB not already promised to other pods, or the pod stays `Pending` with `Insufficient cpu` or `Insufficient memory`.
- `limits`: the ceiling, enforced by the kernel's cgroups. **CPU 3:** the pod's processes get at most 3 CPUs' worth of time; busier than that, they are slowed down (throttled), not killed. **Memory 12 GiB:** above it, the kernel's out-of-memory killer ends a process inside the pod. For RAC that means an instance or Grid process dies, so the size must cover the PGA, the processes, the ASM instance and `/dev/shm`.
- **requests = limits for both CPU and memory** puts the pod in the **Guaranteed** QoS class. If worker1 runs short of memory, Guaranteed pods are the last the kubelet evicts and the last the kernel picks to kill.
- `hugepages-2Mi: 4Gi`: 2048 pages of 2 MiB, which worker1 reserved at boot in guide Phase 4. Hugepages are counted separately from `memory`, and the request is taken to equal the limit. The database SGA goes into them: they're locked in RAM, never swapped, and cheaper for the CPU to map. If the worker hasn't reserved them, the pod stays `Pending` with `Insufficient hugepages-2Mi`.
- **Sizing:** the doc uses 16 GiB, 4 CPUs and 8 GiB of hugepages. The lab workers have 22 GB and 4 vCPUs; 12 + 4 GiB leaves room for the worker's own system, the kubelet and CRI-O, and 3 CPUs leave one for them.

securityContext: capabilities instead of privileged

```yaml
        securityContext:
          privileged: false
          capabilities:
            add: ["NET_ADMIN", "NET_RAW", "SYS_CHROOT", "SYS_NICE", "SYS_RESOURCE", "AUDIT_WRITE",
"AUDIT_CONTROL"]
```

Linux splits root's power into **capabilities**. A `privileged: true` container gets all of them, plus every device on the host and no SELinux or seccomp confinement. That would work, but it hands the pod the whole worker. Instead the pod stays unprivileged and gets only what Grid needs, on top of CRI-O's small default set (`CHOWN`, `DAC_OVERRIDE`, `FOWNER`, `FSETID`, `KILL`, `NET_BIND_SERVICE`, `SETGID`, `SETPCAP`, `SETUID`).

CAPABILITY	LETS THE POD	USED BY
`NET_ADMIN`	change interfaces, addresses and routes	Grid's HAIP puts `169.254.x.x` addresses on `eth1` / `eth2`; Grid's network checks; the image's `ping` carries this file capability
`NET_RAW`	open raw sockets (ICMP, ARP)	`ping`, used by the installer's checks and Grid's network monitoring
`SYS_CHROOT`	call `chroot`	`sshd`'s privilege separation; without it SSH logins fail, and the installers copy everything to node 2 over SSH

CAPABILITY	LETS THE POD	USED BY
`SYS_NICE`	raise priorities and use real-time scheduling	Grid's `ocssd` and agents, and the database's LMS and VKTM processes, run at real-time priority so heartbeats are never late; a late heartbeat can mean an eviction
`SYS_RESOURCE`	go above resource limits	the high `nofile`, `nproc` and `memlock` limits the preinstall package sets for grid and oracle
`AUDIT_WRITE`	write to the kernel audit log	PAM during `ssh` and `su` logins
`AUDIT_CONTROL`	set audit state, including a process's login ID	`pam_loginuid` during `ssh` logins; without it `sshd` can refuse the login

lifecycle.preStop: an orderly shutdown

```yaml
lifecycle:
  preStop: { exec: { command: ["/bin/bash","-c","kill -SIGRTMIN+3 1"] } }
```

- **When:** the kubelet runs this inside the container when the pod is being deleted: `kubectl delete pod`, a template change, scaling to 0, or a node drain.
- **What:** sends signal `SIGRTMIN+3` to process 1, systemd. For systemd that means "halt": stop all services in order (Grid's `oracle-ohasd` among them, then `sshd` and the rest), then exit.
- **Why not the normal way:** after `preStop`, Kubernetes sends `SIGTERM` to process 1. A normal program exits on `SIGTERM`, but systemd as process 1 treats it as "re-execute yourself" and doesn't shut down. Without `preStop`, nothing stops cleanly until the grace period runs out and everything is killed with `SIGKILL`.
- **The grace period** isn't set, so it's Kubernetes' default **30 seconds**, including the `preStop` time. A full Grid and database shutdown can take longer. Then the rest is killed, much like a power cut: the other node recovers the instance, as Phase 14 shows. For planned work, stop Grid in the pod first (`crsctl stop crs` as root), then delete the pod.

volumeDevices: raw ASM disks

```yaml
volumeDevices:
- { name: asmvol1, devicePath: /dev/asm-disk1 }
- { name: asmvol2, devicePath: /dev/asm-disk2 }
```

- **What:** for a claim in `Block` mode, CRI-O creates a **device node** at `devicePath` inside the container and allows the container's cgroup to use that device. Nothing is mounted; ASM reads and writes the raw disk.
- **Why `volumeDevices` and not `volumeMounts`:** a Block claim can only be used this way. Under `volumeMounts`, the pod fails to start.
- **Why fixed names:** Grid's install response file sets the disk discovery string `/dev/asm-disk*` (Phase 11). Both pods see the same disks under the same names, whatever `sdX` letters the workers gave them. The whole chain is in the device-chain guide.

- **Owner:** the node is created as `root:root`. `setupContainerEnv.sh` changes it to `grid:asmadmin` 660 at every start.

readinessProbe: when the pod counts as Ready

```yaml
        readinessProbe:
          tcpSocket: { port: 1521 }
          initialDelaySeconds: 10
          periodSeconds: 30
          failureThreshold: 1
```

- **What:** every 30 seconds, starting 10 seconds after the container starts, the kubelet tries a TCP connection to port 1521 on the pod's IP. If it connects, the pod is Ready (`1/1`); one failure (`failureThreshold: 1`) makes it not Ready (`0/1`).
- **Who listens on 1521:** the SCAN listeners, after Grid is installed (Phase 11). Until then the pod is `Running` but `0/1`, which is normal.
- **Where it's used:**
- `kubectl get pods`, in the READY column.
- Services **without** `publishNotReadyAddresses`, which only send traffic to Ready pods. That's the three NodePort Services of 13.3, so Windows clients aren't sent to a pod whose listeners are down.
- The five headless Services of 10.3 **ignore** readiness, so the names resolve during the install.
- **No `livenessProbe`, on purpose:** a failed liveness probe makes the kubelet **restart** the container. During the install, patching, or a Grid restart, port 1521 is closed for minutes; a liveness probe would keep restarting the RAC node at exactly the wrong time. The readiness probe only reports; it never restarts anything.

volumeMounts: where the directories appear

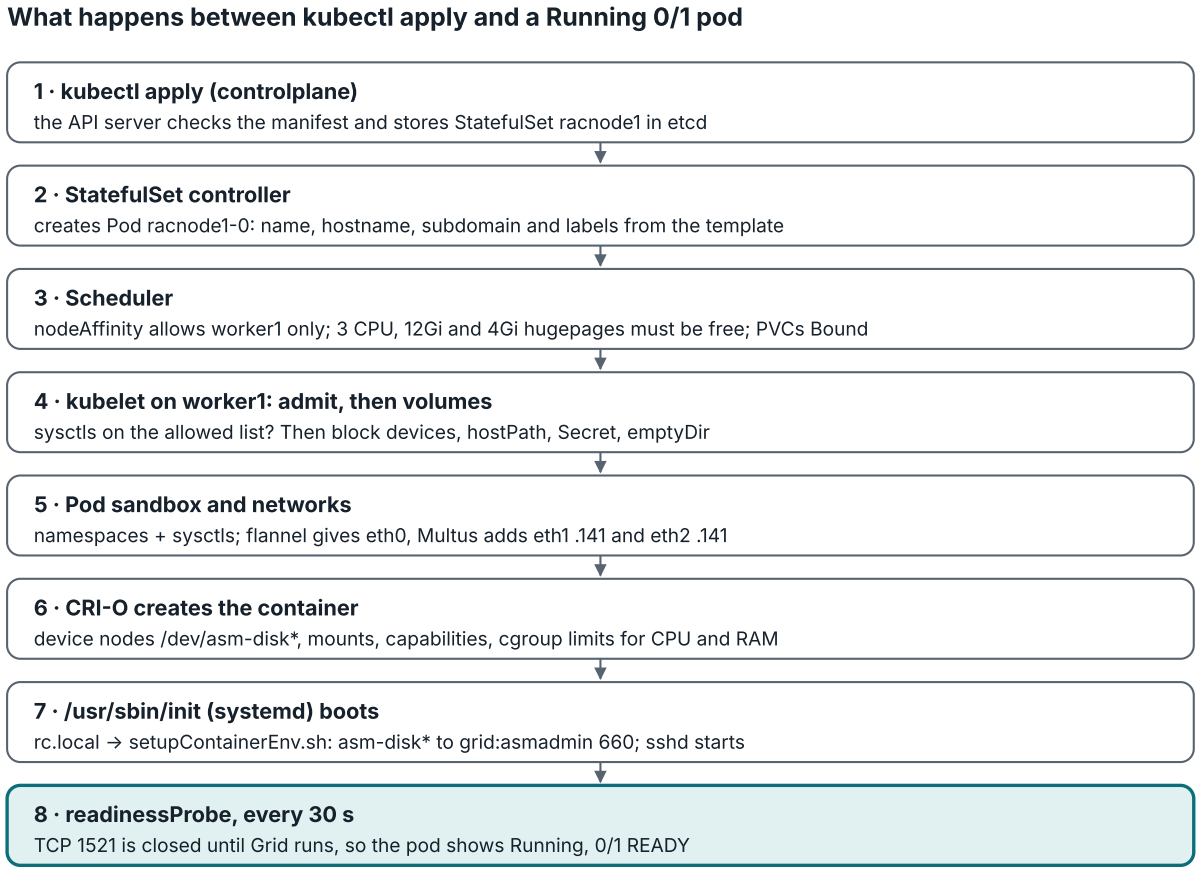
```yaml
        volumeMounts:
        - { mountPath: /dev/shm, name: myshm }
        - { mountPath: /u01, name: swvol-racnode1 }
        - { mountPath: /mnt/.ssh, name: sshkeys, readOnly: true }
        - { mountPath: /stage, name: stagevol }          # node 1 only
```

Each line connects a volume declared above (by `name`) to a path inside the container.

- `/u01`: the Oracle base. Everything the installers write under it lands on worker1's disk.
- `/mnt/.ssh`, `readOnly: true`: the Secret's two files. Read-only because nothing in the pod should change the shared key. The doc's text asks for this, but its YAML doesn't set it.
- `/dev/shm` replaces the 64 MB default with the RAM-backed volume.
- `/stage` exists only in racnode1-0, from the `$stagemnt` line of the shell loop.

What happens when you apply it

`kubectl apply` returns in a second, but the pod is built by several components, one after the other. Knowing the order tells you where to look when a pod doesn't come up.



About a minute for each pod in the lab. Steps 4 to 6 are where most first-build failures show up: kubectl -n rac describe pod says which.

STEP	WHO	USES THESE FIELDS	IF IT FAILS, YOU SEE
1	API server on controlplane	the whole manifest (schema, selector = labels)	an error straight from `kubectl apply`
2	StatefulSet controller	`name`, `serviceName`, `selector`, `template`	`kubectl -n rac get sts` shows `0/1` and no pod; `describe sts` has the reason
3	Scheduler	`affinity`, `resources.requests`, hugepages, the PVCs	`Pending`; `describe pod` says `didn't match Pod's node affinity`, `Insufficient hugepages-2Mi`, or `unbound PersistentVolumeClaims`
4	kubelet on worker1	`sysctls` (against `allowedUnsafeSysctls`), `volumes`	`SysctlForbidden`; `ContainerCreating` with `FailedMount` or `FailedMapVolume` events

STEP	WHO	USES THESE FIELDS	IF IT FAILS, YOU SEE
5	CRI-O sandbox, flannel and Multus	the Multus annotation, `sysctls`	`ContainerCreating` with `FailedCreatePodSandBox` (for example a wrong NetworkAttachmentDefinition name)
6	CRI-O container	`image`, `imagePullPolicy`, `volumeDevices`, `volumeMounts`, `capabilities`, `limits`	`ErrImagePull` or `CreateContainerError`
7	systemd in the container	`command`, then the image's `rc.local`	`CrashLoopBackOff` with no log output: SELinux `container_manage_cgroup` is off on that worker (guide 10.4's note)
8	kubelet, every 30 s	`readinessProbe`	`Running 0/1`: expected until Grid is installed

The events behind every row are in one place:

```
controlplane
CONNECT FIRST  ssh -p 2210 root@127.0.0.1

kubectl -n rac describe pod racnode1-0 | sed -n '/^Events:/,$p'
kubectl -n rac get events --sort-by=.lastTimestamp | tail -20
```

Lifecycle: what survives what

The StatefulSet keeps working long after 10.4: every restart, reboot and change goes through it. This table is the quickest way to predict what a given action does to a RAC node.

EVENT	WHAT KUBERNETES DOES	SAME NAME AND WORKER?	POD IP (ETH 0)	ETH1 / ETH2	/U01 AND ASM DISKS	CONTAINER'S OWN FILES
systemd (process 1) exits or crashes	the kubelet starts a **new container** in the same pod (`RESTARTS` goes up)	yes	same	same	kept	**lost**
`kubectl delete pod racnode1-0`	`preStop`, up to 30 s, then the controller creates a new pod with the same name	yes	usually **new**	same (fixed IPs)	kept	**lost**

EVENT	WHAT KUBERNETES DOES	SAME NAME AND WORKER?	POD IP (`ETH0`)	`ETH1` / `ETH2`	`/U01` AND ASM DISKS	CONTAINER'S OWN FILES
worker1 reboots	when the kubelet is back, it starts the pod's containers again	yes	usually **new**	same	kept	**lost**
worker1 stays down	after about 5 minutes the pod is marked for eviction but can't be confirmed gone, so **no** replacement starts anywhere	(stays on worker1)	-	-	kept	-
`kubectl -n rac scale sts racnode1 --replicas=0`	deletes the pod and doesn't recreate it; `--replicas=1` brings it back	yes, when scaled up	new	same	kept	**lost**
template change (`kubectl apply` with an edit)	rolling update: deletes the pod, creates it from the new template	yes	new	as in the new template	kept	**lost**
`kubectl -n rac delete sts racnode1`	deletes the StatefulSet **and** its pod; re-running 10.4 for node 1 recreates both	yes, after re-apply	new	same	kept	**lost**
`kubectl -n rac delete pvc block-asm-pvc-vol1`	the claim waits in `Terminating` while a pod uses it; afterwards the PV is `Released` and, with `Retain`, the disk's data stays	-	-	-	data on the disk stays; the claim must be recreated by hand	-

What "lost" means for RAC: the files Grid's root scripts wrote outside `/u01` (`/etc/oracle` , the `oracle-ohasd` service, `/etc/oratab` and others). Without them Grid doesn't start in the new container. Guide Phase 14 snapshots those files to `/u01` and restores them automatically at every start, so the node rejoins on its own.

Why a new pod IP doesn't matter: Grid stores the node and VIP **names**, and the headless Services of 10.3 point those names at whatever IP the pod has now. The private IPs, which Grid's interconnect depends on, are fixed by the Multus annotation.

Why "worker1 stays down" leaves node 1 down: a StatefulSet never runs two pods with the same name, and it can't know that the unreachable pod has really stopped. That's the protection RAC needs: the surviving node carries the database, and a second racnode1-0 can't appear on the same disks. The `nodeAffinity` would keep it off worker2 anyway. Don't use `kubectl delete pod --force` while worker1 might still be running.

Try it yourself

These commands only read, except the last block. Run them after guide 10.4, when both pods are `Running`.

The objects, and the defaults Kubernetes filled in:

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac get sts -o wide
kubectl -n rac get pods -o wide --show-labels
for s in racnode1 racnode2; do kubectl -n rac get sts $s -o jsonpath='{.metadata.name}: replicas=
{.spec.replicas} policy={.spec.podManagementPolicy} update={.spec.updateStrategy.type} grace=
{.spec.template.spec.terminationGracePeriodSeconds}s{"\n"}'; done
for p in racnode1-0 racnode2-0; do kubectl -n rac get pod $p -o jsonpath='{.metadata.name}: owner=
{.metadata.ownerReferences[0].kind}/{.metadata.ownerReferences[0].name} node={.spec.nodeName} hostname=
{.spec.hostname} subdomain={.spec.subdomain} qos={.status.qosClass} ip={.status.podIP}{"\n"}'; done
```

Expect: `replicas=1 policy=OrderedReady update=RollingUpdate grace=30s`; each pod owned by its StatefulSet, on its own worker, `subdomain=racnode`, `qos=Guaranteed`.

What Multus attached (Multus writes the result back as an annotation):

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac get pod racnode1-0 -o jsonpath='{.metadata.annotations.k8s\.v1\.cni\.cncf\.io/network-
status}'; echo
```

Expect: three entries: flannel's `eth0` with the pod IP, `eth1` with 10.0.11.141 and `eth2` with 10.0.12.141.

Each field as seen from inside the pod:

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
echo "--- name";        hostname; grep racnode /etc/hosts
echo "--- process 1";   ps -p 1 -o comm=
echo "--- sysctls";     sysctl kernel.shmmax kernel.shmall kernel.shmmni kernel.sem
net.ipv4.conf.all.rp_filter
echo "--- limits";      cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/cpu.max
/sys/fs/cgroup/hugetlb.2MB.max
echo "--- caps";        capsh --decode=$(awk '/CapBnd/{print $2}' /proc/1/status) 2>/dev/null || grep
CapBnd /proc/1/status
echo "--- storage";     ls -l /dev/asm-disk*; df -h /dev/shm /u01 /mnt/.ssh; ls /stage 2>/dev/null
echo "--- networks";    ip -4 -br addr
```

Expect:

- host name `racnode1-0`, and `/etc/hosts` with `racnode1-0.racnode.rac.svc.cluster.local racnode1-0`;
- process 1 is `systemd`;
- the five sysctl values from the manifest;
- `12884901888` (12 GiB), `300000 100000` (3 CPUs: 300 ms in every 100 ms), `4294967296` (4 GiB of hugepages);
- the capabilities list includes `cap_net_admin`, `cap_net_raw`, `cap_sys_chroot`, `cap_sys_nice`, `cap_sys_resource`, `cap_audit_write` and `cap_audit_control`;
- the ASM devices as `grid asmadmin`, mode `brw-rw----`; `/dev/shm` as `tmpfs`; `/u01` on worker1's `/scratch` disk; `/stage` with the zips;
- `eth0`, `eth1` and `eth2` with their addresses.

The worker's side: hugepages, the `/u01` directory and the container:

worker1

CONNECT FIRST `ssh -p 2211 root@127.0.0.1`

```
grep -E 'HugePages_(Total|Free)' /proc/meminfo
ls -la /scratch/rac/cluster01/node1
crictl ps --name racnode1-0
```

Expect: `HugePages_Total: 2048`; the directory that the pod sees as `/u01`; one running container. `HugePages_Free` only drops once the database's SGA uses them.

What the scheduler has reserved on each worker:

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
for w in worker1 worker2; do echo "--- $w"; kubectl describe node $w | sed -n '/Allocated
resources/,/Events/p' | grep -E 'cpu|memory|hugepages'; done
```

Expect: about 3 CPUs, 12 GiB of memory and 4 GiB of `hugepages-2Mi` requested on each worker, mostly by the RAC pod.

Watch the StatefulSet recreate a pod. Only do this **before Grid is installed** (between guide 10.5 and 11), or as part of Phase 14's test. After the install, a pod deletion is a RAC node crash.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac get pod racnode2-0 -o wide
kubectl -n rac delete pod racnode2-0
kubectl -n rac get pod racnode2-0 -o wide -w
```

Expect: the pod goes `Terminating`, then a new `racnode2-0` appears on **worker2** again, usually with a new pod IP, and reaches `Running 0/1`. Press Ctrl+C to stop watching. `/u01` on worker2 and the ASM disks are untouched.

What breaks without each field, and production

Leave one field out and this is what you'd see:

WITHOUT	SYMPTOM	WHEN YOU'D NOTICE
a StatefulSet (a Deployment instead)	a random pod name; Grid sees an unknown node after every restart	the first pod restart after the install
`hostname` in the selector	Kubernetes warns about overlapping selectors; the two StatefulSets each "see" both pods	at once, as a warning
the template labels	the Services of 10.3 and 13.3 select no pod; names don't resolve	Grid's prerequisite checks (Phase 11)
the Multus annotation	no `eth1` / `eth2`; no private interconnect	the Grid installer's network step
`sysctls`	Oracle's defaults are too small: the ASM or database instance fails to allocate its SGA or semaphores	instance start in Phase 11 or 12
`rp_filter: 2`	interconnect packets may be dropped by the kernel	random node evictions, or the installer's interconnect checks
`nodeAffinity`	the pod may start on the other worker with an empty `/u01`	the first restart after the install
`/dev/shm` volume	64 MB `/dev/shm`; ASM's memory target doesn't fit	the Grid install, at ASM start
`/u01` hostPath	Grid and database homes inside the container; all lost at the first restart	the first restart
`volumeDevices`	no ASM disks in the pod	the Grid install's disk discovery

WITHOUT	SYMPTOM	WHEN YOU'D NOTICE
`imagePullPolicy: IfNotPresent`	`ErrImagePull`	at once
`/usr/sbin/init`	no systemd: no `sshd`, no `rc.local`, Grid can't register its service	the pod's first start, then the install
hugepages limit	the pod can't use the worker's hugepages; the SGA uses normal memory and counts against the 12 GiB	database start, or poorer performance
`capabilities`	SSH logins fail, `ping` fails, Grid can't set real-time priorities or HAIP addresses	the SSH setup in 11.1, then the install
`preStop`	systemd ignores `SIGTERM`, so every deletion is a 30-second wait and a hard kill	every pod deletion
`readinessProbe`	the pod always counts as Ready; the NodePorts send clients to a node whose listeners are down	client errors during a node restart

In production you'd keep the design and change these:

- **Shared storage from a CSI driver**, not `local` PVs on VirtualBox disks: a SAN or NVMe-oF array whose driver offers shared Block volumes (`ReadWriteMany` , `volumeMode: Block`), with multipath on the hosts.
- **The Oracle homes on a PV**, not a hostPath, or at least a hostPath with `type: Directory` so that a missing directory fails loudly instead of being silently created empty.
- **A longer** `terminationGracePeriodSeconds` , such as 600, so that a pod deletion gives Grid time to stop. And a `preStop` that first runs `crsctl stop crs` and then halts systemd.
- **Dedicated, tainted nodes** for the RAC pods, with a matching toleration in the template, so nothing else is scheduled next to the database. Plus a namespace with explicit Pod Security exceptions for the capabilities and sysctls.
- **Images from a registry**, with `imagePullPolicy: IfNotPresent` and a fixed tag or digest, instead of building the image on every worker.
- **CPU Manager** `static` **and Topology Manager** on the workers, so the Guaranteed RAC pod gets whole, dedicated CPUs and memory on one NUMA node.
- **A PodDisruptionBudget** with `maxUnavailable: 1` across both RAC pods, so a node drain during maintenance can't take both down at once.
- **Oracle's own operator.** The Oracle Database Operator for Kubernetes (OraOperator) has a RAC controller (`RacDatabase`) that builds the pods, Services and storage from a smaller spec. The needs it has to meet are the ones in this guide, so this guide helps you read what it creates.

Built from the lab's build log and scripts (rac-k8s-poc). Values are from the author's lab; change them to match yours.